Tassonomia

La tassonomia dei Sistemi Operativi è un modo per classificare e comprendere le varie tipologie di software che gestiscono le risorse di un computer. Un S.O. può appartenere a più categorie, anche se alcune si escludono l'una con l'altra (ad esempio monoutente e multiutente).

Un primo criterio di distinzione riguarda il numero di utenti: esistono sistemi progettati per un solo utente (single-user), come quelli tipici dei computer personali, e altri capaci di servire contemporaneamente più utenti (multiuser), come nei server o nei grandi sistemi aziendali.

Un altro aspetto fondamentale è il modo in cui gestiscono l'esecuzione dei programmi: alcuni sistemi possono eseguire un solo compito alla volta (monotasking), mentre altri, più evoluti, permettono di lavorare su più programmi contemporaneamente (multitasking), garantendo un utilizzo più efficiente delle risorse.

In passato, i primi sistemi operativi erano basati su esecuzioni batch: i programmi venivano elaborati a lotti, uno dopo l'altro, senza bisogno di intervento diretto dell'utente. Con l'evoluzione della tecnologia, si sono diffusi i sistemi time-sharing, che consentono un'interazione diretta con l'utente, dividendo il tempo di calcolo tra i vari processi

Esistono poi i sistemi operativi in tempo reale, pensati per applicazioni che richiedono risposte immediate e prevedibili, come nel controllo di impianti industriali o nei dispositivi di bordo degli aerei.

Un altro criterio di classificazione riguarda la distribuzione delle risorse: alcuni sistemi operativi gestiscono risorse concentrate in un'unica macchina (S.O. centralizzati), mentre altri coordinano risorse distribuite su più computer collegati in rete, come accade nei sistemi distribuiti o nelle moderne architetture Cloud.

Infine, i sistemi operativi si distinguono anche in base al tipo di interfaccia offerta all'utente: alcuni richiedono comandi testuali (interfaccia CLI), altri forniscono un'interfaccia grafica (GUI) più intuitiva, come quella a cui siamo abituati su computer e smartphone.

Processo

Un processo è un programma in esecuzione.

Sistemi monoprogrammati (single-task)

Con monoprogrammazione intendiamo la caratteristica di avere solamente un programma in esecuzione. Pertanto, si ha una gestione **sequenziale** dei processi: un processo può essere mandato in esecuzione solamente quando quello precedente è stato completato.

Questo significa che tutte le risorse del sistema sono dedicate al singolo programma che è in esecuzione; da un lato il vantaggio che si ha è la sicurezza, in quanto essendoci un singolo programma in esecuzione, questo non è in grado di accedere allo spazio di altri programmi (perché non ci sono). Dall'altro lato, però, le risorse vengono utilizzate in maniera poco efficiente.

Per capire meglio questo aspetto facciamo un esempio: supponiamo che ci sia un processo in esecuzione che sta utilizzando esclusivamente la CPU, mentre un altro processo in attesa necessita di utilizzare anche una periferica (ad esempio la tastiera). La periferica è libera (l'unico processo in esecuzione non la sta utilizzando), ma il processo che ne ha bisogno non può utilizzarla, in quanto è già presente un altro processo in esecuzione.

Per quanto riguarda l'utilizzazione della CPU, questa è calcolata nel seguente modo:

$$U_{CPU} = \frac{T_p}{T_t}$$

Dove T_p è il tempo dedicato dalla CPU all'esecuzione del programma, e T_t è il tempo totale di permanenza del programma nel sistema.

Possiamo facilmente capire che se T_t è molto grande (il programma rimane tanto nel sistema, ovvero rimane molto tempo in attesa di essere eseguito) e T_p è molto piccolo (il programma è molto veloce), l'utilizzazione della CPU per quel processo è estremamente bassa.

Un'altra caratteristica che possiamo considerare per un sistema di elaborazione è il cosiddetto **throughput**, ovvero il numero di programmi eseguiti nell'unità di tempo.

Abbiamo detto che per i sistemi monoprogrammati la gestione dei processi è sequenziale. Questo è dovuto all'assenza di una caratteristica fondamentale, presente in tutti i sistemi operativi moderni, che è la **preemption**, ovvero la possibilità di revocare forzatamente la CPU ad un processo in esecuzione (interrompendolo) e mandare in esecuzione un altro processo (perché, ad esempio, ha più alta priorità, oppure impiegherebbe molto meno tempo).

Con l'introduzione della preemption anche un sistema monoprogrammato riesce ad avere delle prestazioni migliori, in quanto processi più importanti o che

impiegherebbero meno tempo ad essere eseguiti possono essere mandati in esecuzione interrompendo il processo attualmente eseguito dalla CPU.

Sistemi multiprogrammati

In un sistema multiprogrammato, nella memoria RAM è possibile avere più programmi (oltre al sistema operativo) contemporaneamente. Anche in questo caso solamente un processo alla volta può avere a disposizione la CPU (essere in esecuzione), ma con una differenza rispetto ai sistemi mono-programmati: quando un processo rimane in attesa che si completi una operazione che non richiede la CPU (di solito una operazione di I/O), questa, invece che rimanere inattiva, viene assegnata ad un altro processo secondo un qualche **algoritmo di scheduling**.

Chiaramente le risorse (specialmente la CPU) sono utilizzate in maniera estremamente più efficiente, ma questo porta ad un aumento della complessità del S.O.

Inoltre, sorge un problema che prima non era presente: un processo potrebbe accedere allo spazio di memoria assegnato ad un altro processo (e quindi ai suoi dati). Per evitare questo, il S.O. deve essere sufficientemente potente da impedire che ciò accada; nei moderni sistemi operativi esistono dei meccanismi supportati a livello hardware per impedire che un processo possa accedere allo spazio di indirizzamento non destinato a lui.

Servono anche algoritmi sofisticati per l'assegnazione delle risorse ai diversi processi (che possono richiedere la stessa risorsa contemporaneamente).

Sistemi batch

Batch = lotto.

In un sistema batch i processi vengono raggruppati ed eseguiti in **lotti** (gruppi) in modo da ottenere una migliore utilizzazione delle risorse e, conseguentemente, un throughput più elevato.

La difficoltà in questo tipo di sistemi è capire come raggruppare i diversi processi per ottimizzare l'uso delle risorse.

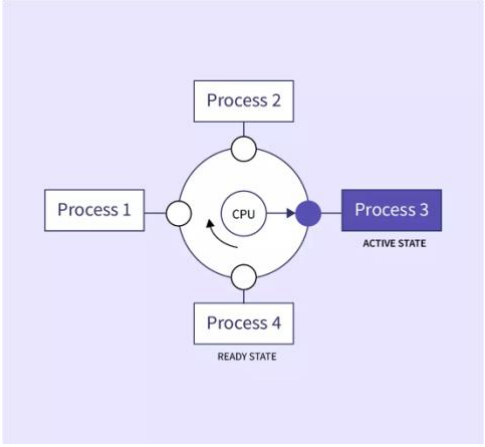
Tipicamente i processi svolgono lunghe e ripetitive elaborazioni, come ad esempio stampe di file di grandi dimensioni, analisi e statistiche sui dati, calcoli e simulazioni scientifiche, …

Di solito Sistemi Operativi di questo tipo sono installati presso grossi e costosi sistemi di elaborazione, presenti in grandi banche, server farm e data center, dove l'obiettivo è sfruttare al massimo le potenti e costose risorse hardware.

Sistemi time-sharing

In un sistema time-sharing sono presenti diversi processi, appartenenti ad utenti diversi che stanno usando il sistema di elaborazione; ad ogni processo viene assegnata la CPU (unica) in maniera esclusiva per un quanto fissato di tempo, chiamato **time slice** (una "fetta" di tempo), tipicamente di circa 10-100 ms.

Se allo scadere del time slice il processo non ha ancora terminato la sua esecuzione, oppure non ha richiesto di eseguire una operazione di I/O, con la quale avrebbe rilasciato spontaneamente la CPU, questa gli viene comunque revocata (preemption) e viene assegnata ad un altro processo in attesa. Osserviamo che, essendo il time slice molto piccolo, ogni processo è convinto di avere sempre a sua disposizione la CPU.



Una gestione di questo tipo migliora i tempi di risposta, ma peggiora l'utilizzazione delle risorse (in quanto anche in questo caso sono ad uso esclusivo di un singolo processo alla volta).

Sistemi multitask

Nei sistemi multiprogrammati abbiamo visto come possono essere presenti più programmi in memoria RAM, in modo tale che quando il processo attualmente in esecuzione richiede una operazione di input/output, la CPU viene rilasciata spontaneamente (yield) e viene assegnata ad un altro processo tra quelli pronti in memoria centrale.

Sorge però un problema: processi **CPU-bound** (o **CPU-intensive**), ovvero processi che hanno principalmente bisogno del processore (perché, ad esempio, devono effettuare molti calcoli) e richiedono poche o nessuna operazione di input/output, non rilasceranno mai il processore o comunque lo occuperebbero per la stragrande maggioranza del tempo, non consentendo a più processi di "evolvere simultaneamente", ovvero avanzare con la propria esecuzione senza interferire con quella degli altri.

Quello che si vuole, è che ogni processo riesca a proseguire con la propria esecuzione, indipendentemente da quella degli altri, avendo l'illusione di avere il processore completamente dedicato. Quindi combiniamo i sistemi multi-programmati con quelli time-sharing; la principale differenza con questi ultimi risiede nel fatto che non è necessario che i processi appartengano ad utenti diversi che stanno usando contemporaneamente il sistema di elaborazione.

Osserviamo come tutti i sistemi operativi per PC più diffusi ed utilizzati (Windows, macOS, Linux) appartengano a questa categoria.

Sistemi real-time

Si può pensare che i sistemi real-time (in tempo reale) siano stati chiamati così in quanto forniscono una risposta all'utente in maniera istantanea. In realtà non è proprio così: un sistema si definisce in tempo reale quando restituisce una risposta all'utente entro un certo tempo prefissato, ovvero ogni processo ha una **deadline** (scadenza) da rispettare.

I sistemi in tempo reale sono principalmente presenti nei cosiddetti sistemi **cyberfisici**, ovvero quei sistemi nei quali è presente una parte informatica che interagisce con una componente fisica.

I sistemi real-time si possono classificare ulteriormente in:

1. **soft real-time**, nei quali è tollerato un piccolo ritardo (se non avviene troppo frequentemente);
2. **hard real-time**, nei quali anche il più piccolo ritardo può essere disastroso.

Un esempio di sistema della prima tipologia è un ATM (Automated Teller Machine), ovvero lo sportello bancomat, in quanto l'operazione deve sì essere effettuata entro un certo tempo, ma un breve ritardo non causa nessun problema.

Un esempio di sistema della seconda tipologia è un sistema di controllo di un velivolo: se il sistema deve calcolare entro un certo tempo la variazione di quota o la modifica alla rotta da apportare all'aereo, anche un lieve ritardo può avere conseguenze disastrose.

Ma cosa succede se la componente di elaborazione, in un sistema a tempo reale, non assicura una risposta in tempo utile agli eventi? Il sistema può diventare potenzialmente pericoloso, e possiamo parlare di sistemi:

- **mission critical**, nei quali un fallimento (una deadline mancata) può portare alla perdita di un obiettivo operativo o una funzionalità essenziale, con gravi conseguenze economiche, operative o strategiche. Ad esempio, possiamo avere una fabbrica/azienda che perde moltissimi soldi perché una risposta ad un evento che non avviene entro un certo tempo ferma l'intera produzione;
- **safety critical**, nei quali un fallimento può causare gravi danni alle persone, ma anche alle infrastrutture o all'ambiente, arrivando a serie lesioni o, in alcuni casi, alla morte. Ancora, possiamo pensare all'esempio del sistema avionico di un aereo, oppure ad un sistema di controllo di un reattore nucleare.

Sistemi monoutente

Nei Sistemi Operativi monoutente, come dice il nome, un solo utente alla volta può utilizzare il sistema. A questa categoria appartengono la maggior parte delle versioni dei classici S.O. utilizzati nei Personal Computer.

A tal proposito occorre precisare una cosa: questi sistemi operativi consentono di creare più profili utente; pensiamo ad esempio a scuola dove possiamo trovare un profilo (account) per ogni studente, per ogni insegnante e anche un profilo amministratore, oppure a casa un computer condiviso può essere predisposto con un account per ogni componente della famiglia. Il fatto di avere a disposizione più account non rende il S.O. multiutente, in quanto solamente uno di questi account può essere attivo ed utilizzato in un certo momento: quindi, è vero che più utenti possono utilizzare lo stesso sistema, ma in momenti diversi.

Sistemi multiutente

A differenza della categoria precedente, questi tipi di S.O. consentono a più utenti in contemporanea di utilizzare il sistema.

Di solito in questi tipi di sistemi troviamo un server e diversi terminali o PC connessi ad esso, che lavorano in modalità desktop remoto: questo significa che all'utente viene offerta una schermata (anche grafica) e la possibilità di svolgere le sue normali operazioni avendo la sensazione che tutto avvenga localmente, sulla macchina che sta usando, quando in realtà tutte le operazioni vengono eseguite dal server ed inviate al terminale.

È chiaro che il sistema server deve essere sufficientemente potente da gestire tutti gli utenti (le macchine) a lui connesso in modo tale che ognuno di loro non percepisca lentezza durante l'uso del sistema.

Sistemi interattivi

Sono S.O. che devono fornire delle risposte in tempi molto brevi, sotto la continua sollecitazione tramite tastiera, mouse, touchscreen o altro da parte dell'utente (umano).

Sistemi embedded

Rientrano in questa categoria tutti quei S.O. che sono pensati e progettati per controllare microprocessori o microcontrollori inclusi/ospitati all'interno di sistemi meccanici e/o elettrici; pensiamo, ad esempio, agli elettrodomestici.

Sistemi transazionali

Comprende quei software che devono essere in grado di gestire in maniera rapida e affidabile delle transazioni.

> **Transazione**
>
> Sequenza di operazioni che devono essere eseguite come una unità indivisibile.

A differenza dei sistemi batch, dove le operazioni da eseguire sono poche, ma molto lunghe, in questi sistemi le operazioni sono moltissime, ma molto brevi.

Le caratteristiche che devono essere soddisfatte da un sistema transazionale sono racchiuse nell'acronimo **ACID**:

- **Atomicità**: un processo deve poter essere suddiviso in unità indivisibili, le transazioni. Questo significa che una transazione deve essere eseguita nella sua totalità, non può avvenire una esecuzione parziale;
- **Consistenza**: i dati gestiti dal sistema transazionale devono essere coerenti sia prima che dopo l'esecuzione della transazione;
- **Isolamento**: l'esecuzione di una transazione è totalmente indipendente dall'esecuzione di altre transazioni. Questo significa che se una transazione fallisce, il fallimento non causa problemi alle altre transazioni;
- **Durabilità**: una volta che la transazione si è completata con successo, le modifiche sono permanenti, non possono essere perse.

Ma cosa succede se una transazione fallisce? Sicuramente deve avvenire la notifica del fallimento della transazione stessa; inoltre, ci deve essere un ripristino automatico del sistema alla situazione consistente precedente.

Questo significa che se la transazione stava modificando dei dati e va in errore, le modifiche devono essere annullate, perché in caso contrario si avrebbe un sistema in uno stato inconsistente (i dati sono cambiati, ma la transazione è stata annullata).

Questa operazione prende il nome di **roolback**.

Sistemi monoprocessore

Sono pensati per gestire una singola CPU.

Sistemi multiprocessore

Sono pensati per poter gestire più di un processore fisico. Sistemi di questo tipo sono molto costosi e si trovano soprattutto in grossi server, mainframe e super computer presenti in datacenter o centri di ricerca.

Osserviamo come la maggior parte dei PC ad uso personale o aziendale siano dotati di un singolo processore (fisico), anche se i sistemi operativi installati (Windows, Linux, macOS) siano in grado di gestire più processori.

Sistemi multicore e sistemi multithread(ed)

Sono sistemi progettati per aumentare il grado di parallelismo nell'esecuzione dei processi, anche in presenza di un unico processore.

Nel corso degli anni è avvenuto un costante rimpicciolimento delle dimensioni dei processori; questo ha consentito di realizzare, su una singola CPU, più **core**, unità di elaborazione indipendenti e autonome, ognuna delle quali funziona esattamente come una CPU.

Dal momento che un processo ha bisogno di una unità di elaborazione (CPU) per essere eseguito, averne di più porta ad un aumento dei processi che è possibile eseguire contemporaneamente (aumenta il parallelismo).

Inoltre, esiste un ulteriore livello di granularità all'interno dei core, in grado di aumentare il parallelismo: stiamo parlando dei cosiddetti **thread**; in sostanza, un thread è una parte di un processo che è possibile eseguire in maniera indipendente dalle altre, consentendo quindi di eseguire più attività contemporaneamente all'interno dello stesso programma (o meglio dire processo).

In sostanza è come se lo stesso flusso di esecuzione di dividesse in flussi più piccoli, eseguibili su core differenti.

Questi sistemi sono quelli maggiormente presenti nei classici computer ad uso personale.

Sistemi distribuiti

I Sistemi Operativi distribuiti sono una categoria di S.O. progettati per gestire un insieme di elaboratori interconnessi tramite la rete (cluster), facendoli apparire come un'unica macchina agli occhi dell'utente.

Quello che fa un S.O. di questo tipo è coordinare le risorse hardware e software che non risiedono su un solo computer, ma su più nodi all'interno della rete, in maniera del tutto trasparente, garantendo i classici servizi di un Sistema Operativo (gestione dei file, gestione dei processi, …).

Quindi una caratteristica fondamentale che deve essere garantita da un S.O. distribuito è la **trasparenza**, la quale si declina in diversi aspetti:

- **trasparenza di accesso**: l'utente accede alle risorse remote nella stessa identica maniera con la quale accede a quelle locali, sulla propria macchina, senza notare differenze;

- **trasparenza di locazione**: l'utente non deve essere a conoscenza della posizione fisica di una risorsa all'interno del sistema;
- **trasparenza di replica**: può capitare che una o più risorse vengano duplicate su più nodi della rete per maggiore efficienza o per ragioni di affidabilità, ma l'utente non deve accorgersi che questo avviene;
- **trasparenza di concorrenza**: più utenti o processi possono accedere alle stesse risorse senza che si verifichino conflitti;
- **trasparenza al guasto**: detta anche **fault tolerance,** consiste nel nascondere un guasto che avviene nel sistema, mantenendo quest'ultimo correttamente funzionante.